

กลุ่มที่ 4 Expression Processor: Infix และ Postfix

1. วิเคราะห์ปัญหา:

โจทย์: รับนิพจน์คณิตศาสตร์แบบ Infix เช่น “ $3 + 4 * 2$ ” เข้ามา ตรวจสอบว่าถูกไวยากรณ์หรือไม่ ถ้าถูกต้องให้แปลงเป็น Postfix แล้วคำนวณผลลัพธ์ออกมา

Input:

- String นิพจน์ที่มี Operator + - */ และวงเล็บ
- ตัวเลขเป็นจำนวนเต็มบวก มีได้หลายหลัก เช่น 123
- มีช่องว่างคั่นระหว่าง token ได้

Output:

- ถ้านิพจน์ถูกต้อง: Postfix Expression และผลลัพธ์ตัวเลข
- ถ้าผิด: แจ้ง error ว่าผิดตรงไหน

ข้อจำกัด:

- รองรับเฉพาะ + - * / กับวงเล็บกลมเท่านั้น
- เลขเป็นจำนวนเต็มบวกล้วน ไม่มีทศนิยม
- ห้ามหารด้วย 0

ข้อสมมติฐาน:

- input เป็น string ที่อ่านทีละ token ได้ชัดเจน
- operator มี priority ตามหลักคณิตศาสตร์ปกติ: * / มาก่อน + -

กรณีปกติ:

- $3 + 4 * 2$ (คำนวณตาม Priority ปกติ)
- $(3 + 4) * 2$ (วงเล็บเปลี่ยนลำดับ)

กรณีขอบเขต:

- นิพจน์ว่าง
- นิพจน์ที่มีตัวเลขตัวเดียวเช่น 5
- วงเล็บซ้อนกันหลายชั้น เช่น $((8 + 2) * 5)$

กรณีที่ทำให้เกิด Error ได้:

- วงเล็บไม่ครบคู่ เช่น $(3 + 4$
- Operator ซ้อนกันโดยไม่มี operand คั่น เช่น $3 + * 4$
- หารด้วยศูนย์ เช่น $10 / (5 - 5)$

Operation ที่เกิดขึ้นบ่อยที่สุด:

- push/pop operand และ operator ระหว่างสแกนแต่ละ token

Operation ที่อาจใช้เวลามากที่สุด:

- ตอนเจอวงเล็บปิด “)” ต้อง pop operator ออกมาคำนวณจนกว่าจะเจอวงเล็บเปิด “(“

2. ออกแบบAlgorithm:

Algorithm A: Infix to Postfix แล้ว Evaluate แยกอีกที

- คือแปลง Infix เป็น Postfix ก่อนทั้งหมด แล้วค่อยนำ Postfix ไปคำนวณผลลัพธ์แยกกันอีกขั้นตอน

Pseudocode A ส่วนที่ 1 แปลง Infix เป็น Postfix:

ALGORITHM InfixToPostfix(expression)

Create an empty operatorStack

Create an empty postfixList

tokens $\leftarrow$ Tokenize(expression)

FOR each token IN tokens

IF token is a number

 Add token to postfixList

ELSE IF token is "("

 Push token to operatorStack

ELSE IF token is ")"

 WHILE operatorStack is not empty AND operatorStack.peek() is not "("

 Pop operator from operatorStack, add to postfixList

 END WHILE

IF operatorStack is empty

 RETURN error "วงเล็บไม่ครบคู่"

END IF

Pop "(" from operatorStack

ELSE IF token is an operator (+, -, *, /)

 WHILE operatorStack is not empty

 AND operatorStack.peek() is not "("

 AND Priority(operatorStack.peek()) $\geq$ Priority(token)

 Pop operator from operatorStack, add to postfixList

 END WHILE

 Push token to operatorStack

ELSE

 RETURN error "token ไม่ถูกต้อง"

```

    END IF
END FOR

WHILE operatorStack is not empty
    IF operatorStack.peek() is "("
        RETURN error "วงเล็บไม่ครบคู่"
    END IF
    Pop operator from operatorStack, add to postfixList
END WHILE

RETURN postfixList

```

Pseudocode A **ส่วนที่ 2 คำนวณค่า** Postfix:

```

ALGORITHM EvaluatePostfix(postfixList)

Create an empty operandStack

FOR each token IN postfixList
    IF token is a number
        Push token to operandStack
    ELSE IF token is an operator
        IF operandStack has fewer than 2 elements
            RETURN error "นิพจน์ไม่ถูกต้อง"
        END IF
        b ← operandStack.pop()
        a ← operandStack.pop()
        IF token is "/" AND b = 0

```

```

    RETURN error "หารด้วยศูนย์"

END IF

result ← Calculate(a, token, b)

Push result to operandStack

END IF

END FOR

IF operandStack has exactly 1 element
    RETURN operandStack.pop()
ELSE
    RETURN error "นิพจน์ไม่ถูกต้อง"
END IF

```

Algorithm B: ประเมิน Infix ตรงๆ ด้วย 2 Stacks

- คือ ใช้ Operand Stack และ Operator Stack ประเมินค่านิพจน์ Infix ไปพร้อมกัน
ในรอบเดียว โดยไม่ต้องแปลงเป็น Postfix ก่อน

Pseudocode B:

```

ALGORITHM EvaluateInfixDirect(expression)

```

```

    Create an empty operandStack

```

```

    Create an empty operatorStack

```

```

    tokens ← Tokenize(expression)

```

```

    FOR each token IN tokens

```

```

        IF token is a number

```

```

            Push token to operandStack

```

ELSE IF token is "("

Push token to operatorStack

ELSE IF token is ")"

WHILE operatorStack is not empty AND operatorStack.peek() is not "("

ApplyTopOperator(operandStack, operatorStack)

END WHILE

IF operatorStack is empty

RETURN error "วงเล็บไม่ครบคู่"

END IF

Pop "(" from operatorStack

ELSE IF token is an operator (+, -, *, /)

WHILE operatorStack is not empty

AND operatorStack.peek() is not "("

AND Priority(operatorStack.peek()) >= Priority(token)

ApplyTopOperator(operandStack, operatorStack)

END WHILE

Push token to operatorStack

ELSE

RETURN error "token ไม่ถูกต้อง"

END IF

END FOR

WHILE operatorStack is not empty

IF operatorStack.peek() is "("

RETURN error "วงเล็บไม่ครบคู่"

END IF

ApplyTopOperator(operandStack, operatorStack)

END WHILE

IF operandStack has exactly 1 element

 RETURN operandStack.pop()

ELSE

 RETURN error "นิพจน์ไม่ถูกต้อง"

END IF

ข้อดีและข้อจำกัดของแต่ละวิธี

	Algorithm A	Algorithm B
ข้อดี	แยก postfix ออกมาดูแลได้ ตรวจสอบ debug ง่าย, evaluate เข้าใจโดยไม่ต้อง parse ใหม่	ประหยัดหน่วยความ จำกว่าเล็กน้อย, โค้ด สั้นกว่าเพราะทำรอบ เดียวจบ
ข้อจำกัด	ต้องเก็บ postfix list เพิ่มอีก ก้อนหนึ่ง	ไม่มี postfix เก็บไว้ดู debug ยากกว่าเล็กน้อย

ตัวอย่าง Step-by-step:

ตัวอย่างที่ 1 (กรณีปกติ) - Algorithm A: $3 + 4 * 2 / (1 - 5)$

Step	Token	Action	Operator Stack	Postfix สะสม
1	3	ตัวเลข -> ใส่ postfix	(ว่าง)	3
2	+	stack ว่าง -> push	+	3
3	4	ตัวเลข -> ใส่ postfix	+	3 4

4	*	priority(*) > priority(+) -> push	* +	3 4
5	2	ตัวเลข -> ใส่ postfix	* +	3 4 2
6	/	priority เท่ากับ * -> pop * ก่อน แล้ว push /	/ +	3 4 2 *
7	(	push เลย	(/ +	3 4 2 *
8	1	ตัวเลข -> ใส่ postfix	(/ +	3 4 2 * 1
9	-	top คือ (-> push เลย	- (/ +	3 4 2 * 1
10	5	ตัวเลข -> ใส่ postfix	- (/ +	3 4 2 * 1 5
11	)	pop จนเจอ (แล้วทิ้ง (	/ +	3 4 2 * 1 5-
12	(จบ)	pop ที่เหลือ ทั้งหมด	(ว่าง)	3 4 2 * 1 5 - / +

ผลลัพธ์ Postfix:

- 3 4 2 * 1 5 - / + จากนั้นคำนวณ: $4 * 2 = 8$, $1 - 5 = -4$, $8 / -4 = -2$, $3 + (-2) = 1$

- ผลลัพธ์สุดท้ายคือ 1

ตัวอย่างที่ 2 (กรณีขอบเขต - วงเล็บซ้อน) - Algorithm B: $((8 + 2) * 5)$

Step	Token	Action	Operator Stack	Operand Stack
1	(	push	(	(ว่าง)

2	(	push	((	(ว่าง)
3	8	push operand	((	8
4	+	top คือ (-> push เลย	+ ((	8
5	2	push operand	+ ((	8 2
6	)	pop + คำนวณ 8 + 2 = 10, push, pop (ทิ้ง	(	10
7	*	top คือ (-> push เลย	* (	10
8	5	push operand	* (	10 5
9	)	pop * มา คำนวณ 10 * 5 = 50, push, pop (ทิ้ง	(ว่าง)	50
10	(จบ)	operand stack เหลือตัว เดียว -> return	-	50

ผลลัพธ์:

= 50 (ตรวจสอบ manual: $(8 + 2) * 5 = 10 * 5 = 50$ ถูกต้อง) วงเล็บซ้อนกัน 2 ชั้น ทำให้ operation stack มี (ค้างอยู่พร้อมกัน 2 ตัว และทุกครั้งที่เจอ) จะปิดการคำนวณของวงเล็บชั้นในสุดก่อนเสมอ ก็เลยสะท้อนว่า stack เหมาะกับปัญหานี้เพราะจัดการโครงสร้างที่ซ้อนกันแบบ LIFO ได้พอดี

3. อธิบายความถูกต้อง:

เลือก Algorithm A (InfixToPostfix) ด้วยแนวทาง Loop Invariant

Invariant

ก่อนเริ่มแต่ละรอบของ loop (แต่ละ token ที่อ่าน) operator ทุกตัวที่อยู่ใน operatorStack เรียงจาก top ลงไปจะมี priority ไม่เพิ่มขึ้น (non-increasing) นับตั้งแต่ตัวบนวงเล็บล่าสุดขึ้นมา และ postfixList ที่สะสมไว้แล้วเป็นลำดับ Postfix ที่ถูกต้องของ token ทั้งหมดที่อ่านผ่านมาแล้ว

พูดง่ายๆ คือ ทุกครั้งที่หยุดดู stack กลางคัน จะไม่มีทางเจอ operator priority ต่ำอยู่ใต้ operator priority สูงกว่าในวงเล็บเดียวกัน และ postfix ที่ print ไปแล้ว จะไม่มีการย้อนกลับมาแก้อีก

Initialization:

ก่อนเริ่ม loop เริ่มรอบแรก operatorStack และ postfixList ยังว่างเปล่าทั้งคู่ ถือว่า invariant เป็นจริงโดยอัตโนมัติ

Maintenance:

- ถ้าเป็นตัวเลข: ใส่ postfixList ตรงๆ ไม่กระทบ stack เลย invariant ยังจริง
- ถ้าเป็น operator ใหม่: ก่อน push จะ pop operator ที่ priority สูงกว่าหรือเท่ากัน ออกมาใส่ postfix ก่อนเสมอ พอ push ตัวใหม่ลงไปมันจะน้อยกว่าหรือเท่ากับตัวข้างล่างเสมอ
- ถ้าเป็น (: push ตรงๆ ถือเป็นขอบเขตใหม่ที่ตัดกลุ่ม priority ไม่ให้ปนกับข้างนอก

นอก

- ถ้าเป็น): pop operator ออกมาจนเจอ (แล้วทิ้ง (นั้นไป เป็นการปิดกลุ่มที่เปิดไว้ ไม่ทำให้ลำดับที่เหลือผิดเพี้ยน

ทุกกรณี operator ที่ pop ออกมาใส่ postfixList จะถูกส่งไปก่อน operator ที่ priority ต่ำกว่าเสมอ ตรง กับกฎ Postfix ที่ operator priority สูงกว่าต้องถูกคำนวณก่อน

Termination:

หลัง loop จบ operator ที่เหลือค้างใน stack จะถูก pop ออกมาใส่ postfixList ทั้งหมด ตาม invariant คือเรียงจาก priority มากไปน้อยจาก top อยู่แล้ว พอ pop ออกมาตามลำดับนี้ก็ยังถูกต้องตามกฎ Postfix ผลคือ postfixList สุดท้ายเป็นลำดับ Postfix ที่ถูกต้องสมบูรณ์ของนิพจน์ทั้งหมด

สรุปตามหัวข้อทั้ง 3:

- ก่อนเริ่ม: stack และ postfix list วางเปล่า ไม่มีอะไรขัดแย้ง
- ระหว่างทำงาน: priority ใน operatorStack เรียงจากมากไปน้อยเสมอในขอบเขตวงเล็บเดียวกัน postfix ที่ print ไปแล้วไม่ต้องแก้อ่อนหลัง
- เมื่อหยุด: operator ที่เหลือถูก pop ออกมาครบตามลำดับ priority ผลลัพธ์

ตรงตามกฎ Postfix

- ข้อมูลไม่สูญหาย: ทุก token ถูกอ่านตามลำดับเดิม และ operator ที่นำออกจาก stack จะถูกย้ายไปยัง

Postfix ตาม priority และวงเล็บ

- หยุดทำงานเสมอ: loop ควบคุมด้วยจำนวน token ที่มีขอบเขตจำกัด จึงไม่มี infinite loop และกรณี error เช่นวงเล็บไม่ครบจะถูกตรวจพบระหว่างการประมวลผลหรือหลังจบการสแกน

4. การวิเคราะห์ประสิทธิภาพ:

ให้ n = จำนวน token ทั้งหมดในนิพจน์ (ตัวเลข + operator + วงเล็บ)

แนวคิดหลัก:

ทั้ง Algorithm A และ B อ่าน token ตามลำดับ และการ push/pop ที่เกิดขึ้นรวมกันมีจำนวนเป็นสัดส่วนกับจำนวน token ดังนั้นแม้จะมี while loop ซ่อนอยู่ ภาพรวมยังเป็น $O(n)$

Algorithm A:

- ขั้น 1 (Infix -> Postfix): อ่าน token ทีละตัว $O(n)$ และ operator ที่ถูก pop ออกมารวมทั้งหมดไม่เกินจำนวน token จึงเป็น $O(n)$
- ขั้น 2 (Evaluate Postfix): อ่าน postfix list หนึ่งรอบ และการทำงานกับ operand stack รวมเป็น $O(n)$
- รวม Algorithm A: $O(n) + O(n) = O(n)$

Algorithm B:

- อ่าน token ทีละตัว $O(n)$ และจัดการ operand stack กับ operator stack ภายในรอบเดียว จึงรวมเป็น $O(n)$
- รวม: $O(n)$

ตารางสรุป Complexity:

Algorithm	Best Case	Average Case	Worst Case	Auxiliary Space
A (Convert + Evaluate)	$O(n)$	$O(n)$	$O(n)$	$O(n)$
B (Direct Evaluate)	$O(n)$	$O(n)$	$O(n)$	$O(n)$

หมายเหตุ: ทั้งคู่ไม่มี best/average/worst ที่ต่างกันจริงๆ เพราะทุก token ต้องถูกอ่านอย่างน้อย 1 ครั้งเสมอไม่ว่ากรณีไหน จึงเป็น $O(n)$ คงที่ทุกกรณี

วิเคราะห์แยกฟังก์ชัน:

ฟังก์ชัน / ขั้นตอน	Complexity	เหตุผล
Tokenize(expression)	$O(n)$	อ่าน string ทีละตัวอักษร ครั้งเดียว

push() / pop() / peek()	$O(1)$	operation พื้นฐานของ stack
Priority(operator)	$O(1)$	เทียบค่าคงที่ ไม่ loop
WHILE loop เทียบ priority	$O(1)$ amortized ต่อ token, รวม $O(n)$	operator แต่ละตัว pop ได้แค่ครั้งเดียว
Calculate(a, op, b)	$O(1)$	คำนวณเลขพื้นฐาน
ตรวจสอบวงเล็บครบคู่	รวมอยู่ใน $O(n)$ เดียวกัน	ทำระหว่าง scan token ไปด้วย

Auxiliary Space:

- Algorithm A: operatorStack + postfixList ระหว่างแปลง แล้วใช้ operandStack ตอน evaluate กรณีเลวร้ายสุด operator ทั้งหมดค้างใน stack พร้อมกัน (เช่นวงเล็บซ้อนเยอะๆ) เป็น $O(n)$
- Algorithm B: operandStack + operatorStack พร้อมกันตลอด เป็น $O(n)$ เช่นกัน แต่ในทางปฏิบัติอาจใช้หน่วยความจำรวมน้อยกว่าเล็กน้อยเพราะไม่ต้องเก็บ postfixList แยกต่างหาก

สรุปเปรียบเทียบ A vs B:

- Time Complexity เท่ากัน ทั้งคู่ $O(n)$ เพราะหลักการเดียวกัน
- ต่างกันที่ Space ในทางปฏิบัติ: A ต้องเก็บ postfix list เพิ่มอีกก้อนหนึ่ง ส่วน B คำนวณสดไปเลยไม่ต้องเก็บ
- ข้อดี A: แยก postfix ออกมาดูได้ debug ง่ายกว่า evaluate ซ้ำได้โดยไม่ต้อง parse ใหม่
- ข้อดี B: ประหยัดหน่วยความจำกว่าเล็กน้อย โค้ดสั้นกว่าเพราะทำรอบเดียวจบ

Test Cases:

Test ID	Input	ประเภท	Expected Postfix	Expected Result
TC01	$3 + 4 * 2$	Normal case	$3\ 4\ 2\ *\ +$	11
TC02	$(3 + 4) * 2$	Normal case	$3\ 4\ +\ 2\ *$	14
TC03	$((8 + 2) * 5)$	Boundary (วงเล็บซ้อน)	$8\ 2\ +\ 5\ *$	50
TC04	$(3 + 4$	Invalid (วงเล็บไม่ครบ)	-	Error: วงเล็บไม่ครบคู่
TC05	$3 + 4)$	Invalid (วงเล็บเกิน)	-	Error: วงเล็บไม่ครบคู่
TC06	$3 + * 4$	Invalid (operator ซ้อน)	-	Error: นิพจน์ไม่ถูกต้อง
TC07	$10 / (5 - 5)$	Invalid (หารศูนย์)	$10\ 5\ 5\ -\ /\$	Error: หารด้วยศูนย์
TC08	(นิพจน์ว่าง)	Empty input	-	Error: นิพจน์ว่างเปล่า
TC09	5	Best case	5	5
TC10	$3 + 4 * 2 / (1 - 5)$	Required Example	$3\ 4\ 2\ *\ 1\ 5\ -\ /\ +$	1

หมายเหตุ: ให้รันโปรแกรมจริงแล้วกรอกผลลัพธ์ที่ได้จริงเทียบกับ

Expected Result เพื่อใช้เป็นหลักฐานการทดสอบ (Actual Result / Status)

6. โปรแกรมภาษา Java:

โครงสร้างโปรแกรมประกอบด้วย Main Class, Data Model Class (models), Algorithm Class (algorithms) และ Utility Class (utils) แยกกันตามหน้าที่ ดังนี้

Tokenizer.java:

แยกนิพจน์ที่รับเข้ามาเป็น token (ตัวเลข, operator, วงเล็บ) และมีฟังก์ชันช่วยเช็ค priority ของ operator

```
package utils;

import java.util.ArrayList;
import java.util.List;

public class Tokenizer {

    public static List<String> tokenize(String expression) throws IllegalArgumentException {
        List<String> tokens = new ArrayList<>();
        int i = 0;
        int n = expression.length();

        while (i < n) {
            char c = expression.charAt(i);

            if (Character.isWhitespace(c)) {
                i++;
                continue;
            }

            if (Character.isDigit(c)) {
                StringBuilder num = new StringBuilder();
                while (i < n && Character.isDigit(expression.charAt(i))) {
                    num.append(expression.charAt(i));
                    i++;
                }
                tokens.add(num.toString());
                continue;
            }

            if (c == '+' || c == '-' || c == '*' || c == '/' || c == '(' || c == ')') {
                tokens.add(String.valueOf(c));
            }
        }
    }
}
```

```

        i++;
        continue;
    }

    throw new IllegalArgumentException("พบตัวอักษรที่ไม่รองรับ: " + c + "");
}

return tokens;
}

public static boolean isOperator(String token) {
    return token.equals("+") || token.equals("-") || token.equals("*") || token.equals("/");
}

public static boolean isNumber(String token) {
    if (token == null || token.isEmpty()) return false;
    for (char c : token.toCharArray()) {
        if (!Character.isDigit(c)) return false;
    }
    return true;
}

public static int priority(String operator) {
    switch (operator) {
        case "+":
        case "-":
            return 1;
        case "*":
        case "/":
            return 2;
        default:
            return 0;
    }
}
}

```

OperationCounter.java:

เก็บจำนวนครั้งของ push, pop, และการเปรียบเทียบ ใช้สำหรับวิเคราะห์ประสิทธิภาพ

```

package models;

public class OperationCounter {

```



```

private long pushCount;
private long popCount;
private long comparisonCount;

public void incrementPush() {
    pushCount++;
}

public void incrementPop() {
    popCount++;
}

public void incrementComparison() {
    comparisonCount++;
}

public long getPushCount() {
    return pushCount;
}

public long getPopCount() {
    return popCount;
}

public long getComparisonCount() {
    return comparisonCount;
}

@Override
public String toString() {
    return "Push=" + pushCount + ", Pop=" + popCount + ", Comparisons=" + comparisonCount;
}
}

```

ExpressionResult.java:

เก็บผลลัพธ์การประมวลผล ทั้งกรณีสำเร็จและกรณี error พร้อมเวลาที่ใช้
และตัวนับ operation

```

package models;

public class ExpressionResult {
    private boolean success;

```

```

private double value;
private String postfix;
private String errorMessage;
private long elapsedTimeNanos;
private OperationCounter counter;

public static ExpressionResult success(double value, String postfix, long elapsedTimeNanos, OperationCounter counter) {
    ExpressionResult r = new ExpressionResult();
    r.success = true;
    r.value = value;
    r.postfix = postfix;
    r.elapsedTimeNanos = elapsedTimeNanos;
    r.counter = counter;
    return r;
}

public static ExpressionResult error(String message, long elapsedTimeNanos, OperationCounter counter) {
    ExpressionResult r = new ExpressionResult();
    r.success = false;
    r.errorMessage = message;
    r.elapsedTimeNanos = elapsedTimeNanos;
    r.counter = counter;
    return r;
}

public boolean isSuccess() { return success; }
public double getValue() { return value; }
public String getPostfix() { return postfix; }
public String getErrorMessage() { return errorMessage; }
public long getElapsedTimeNanos() { return elapsedTimeNanos; }
public OperationCounter getCounter() { return counter; }
}

```

AlgorithmA.java:

แปลง Infix เป็น Postfix ก่อน แล้วค่อย evaluate แยกอีกขั้นตอน ตรงตาม Pseudocode ที่ออกแบบไว้

```

package algorithms;

import models.ExpressionResult;
import models.OperationCounter;
import utils.Tokenizer;

```

```

import java.util.ArrayDeque;
import java.util.ArrayList;
import java.util.Deque;
import java.util.List;

public class AlgorithmA {

    public ExpressionResult evaluate(String expression) {
        OperationCounter counter = new OperationCounter();
        long startTime = System.nanoTime();

        try {
            List<String> tokens = Tokenizer.tokenize(expression);
            List<String> postfix = infixToPostfix(tokens, counter);
            double result = evaluatePostfix(postfix, counter);
            long elapsed = System.nanoTime() - startTime;
            return ExpressionResult.success(result, String.join(" ", postfix), elapsed, counter);
        } catch (IllegalArgumentException | ArithmeticException e) {
            long elapsed = System.nanoTime() - startTime;
            return ExpressionResult.error(e.getMessage(), elapsed, counter);
        }
    }

    private List<String> infixToPostfix(List<String> tokens, OperationCounter counter) {
        Deque<String> operatorStack = new ArrayDeque<>();
        List<String> postfixList = new ArrayList<>();

        for (String token : tokens) {
            if (Tokenizer.isNumber(token)) {
                postfixList.add(token);
            } else if (token.equals("(")) {
                operatorStack.push(token);
                counter.incrementPush();
            } else if (token.equals(")")) {
                while (!operatorStack.isEmpty() && !operatorStack.peek().equals("(")) {
                    counter.incrementComparison();
                    postfixList.add(operatorStack.pop());
                    counter.incrementPop();
                }
                if (operatorStack.isEmpty()) {
                    throw new IllegalArgumentException("วงเล็บไม่ครบคู่");
                }
            }
        }
    }
}

```

```

        operatorStack.pop();
        counter.incrementPop();
    } else if (Tokenizer.isOperator(token)) {
        while (!operatorStack.isEmpty()
            && !operatorStack.peek().equals("(")
            && Tokenizer.priority(operatorStack.peek()) >= Tokenizer.priority(token)) {
            counter.incrementComparison();
            postfixList.add(operatorStack.pop());
            counter.incrementPop();
        }
        operatorStack.push(token);
        counter.incrementPush();
    } else {
        throw new IllegalArgumentException("token ไม่ถูกต้อง: " + token);
    }
}

while (!operatorStack.isEmpty()) {
    if (operatorStack.peek().equals("(")) {
        throw new IllegalArgumentException("วงเล็บไม่ครบคู่");
    }
    postfixList.add(operatorStack.pop());
    counter.incrementPop();
}

return postfixList;
}

private double evaluatePostfix(List<String> postfix, OperationCounter counter) {
    Deque<Double> operandStack = new ArrayDeque<>();

    for (String token : postfix) {
        if (Tokenizer.isNumber(token)) {
            operandStack.push(Double.parseDouble(token));
            counter.incrementPush();
        } else if (Tokenizer.isOperator(token)) {
            double b = operandStack.pop();
            counter.incrementPop();
            double a = operandStack.pop();
            counter.incrementPop();

            double result = calculate(a, token, b);
            operandStack.push(result);
        }
    }
}

```

```

        counter.incrementPush();
    }
}

return operandStack.pop();
}

private double calculate(double a, String operator, double b) {
    switch (operator) {
        case "+": return a + b;
        case "-": return a - b;
        case "*": return a * b;
        case "/":
            if (b == 0) {
                throw new ArithmeticException("หารด้วยศูนย์");
            }
            return a / b;
        default:
            throw new IllegalArgumentException("operator ไม่ถูกต้อง: " + operator);
    }
}
}

```

```

เลือกเมนู: 1
ป้อนนิพจน์ (Infix): 3 + 4 * 2 / (1 - 5)
--- Algorithm A ---
Postfix: 3 4 2 * 1 5 - / +
ผลลัพธ์: 1.0
เวลาที่ใช้: 2181200 ns
จำนวน Operation: Push=14, Pop=14, Comparisons=6, Loops: 20

```

AlgorithmB.java:

ประเมินค่า Infix ตรงๆ ด้วย Operand Stack และ Operator Stack พร้อมกันในรอบเดียว

```

package algorithms;

import models.ExpressionResult;
import models.OperationCounter;
import utils.Tokenizer;

import java.util.ArrayDeque;
import java.util.Deque;

```

```

import java.util.List;

public class AlgorithmB {

    public ExpressionResult evaluate(String expression) {
        OperationCounter counter = new OperationCounter();
        long startTime = System.nanoTime();

        try {
            List<String> tokens = Tokenizer.tokenize(expression);
            double result = evaluateInfix(tokens, counter);
            long elapsed = System.nanoTime() - startTime;
            return ExpressionResult.success(result, "ไม่ได้สร้าง Postfix ใน Algorithm B", elapsed, counter);
        } catch (IllegalArgumentException | ArithmeticException e) {
            long elapsed = System.nanoTime() - startTime;
            return ExpressionResult.error(e.getMessage(), elapsed, counter);
        }
    }

    private double evaluateInfix(List<String> tokens, OperationCounter counter) {
        Deque<Double> operandStack = new ArrayDeque<>();
        Deque<String> operatorStack = new ArrayDeque<>();

        for (String token : tokens) {
            if (Tokenizer.isNumber(token)) {
                operandStack.push(Double.parseDouble(token));
                counter.incrementPush();
            } else if (token.equals("(")) {
                operatorStack.push(token);
                counter.incrementPush();
            } else if (token.equals(")") {
                while (!operatorStack.isEmpty() && !operatorStack.peek().equals("(")) {
                    counter.incrementComparison();
                    applyTopOperator(operandStack, operatorStack, counter);
                }
                if (operatorStack.isEmpty()) {
                    throw new IllegalArgumentException("วงเล็บไม่ครบคู่");
                }
                operatorStack.pop();
                counter.incrementPop();
            } else if (Tokenizer.isOperator(token)) {
                while (!operatorStack.isEmpty()
                    && !operatorStack.peek().equals("(")

```

```

        && Tokenizer.priority(operatorStack.peek()) >= Tokenizer.priority(token)) {
            counter.incrementComparison();
            applyTopOperator(operandStack, operatorStack, counter);
        }
        operatorStack.push(token);
        counter.incrementPush();
    } else {
        throw new IllegalArgumentException("token ไม่ถูกต้อง: " + token);
    }
}

while (!operatorStack.isEmpty()) {
    if (operatorStack.peek().equals("(")) {
        throw new IllegalArgumentException("วงเล็บไม่ครบคู่");
    }
    applyTopOperator(operandStack, operatorStack, counter);
}

return operandStack.pop();
}

private void applyTopOperator(Deque<Double> operandStack, Deque<String> operatorStack, OperationCounter counter)
{
    String op = operatorStack.pop();
    counter.incrementPop();
    double b = operandStack.pop();
    counter.incrementPop();
    double a = operandStack.pop();
    counter.incrementPop();

    double result = calculate(a, op, b);
    operandStack.push(result);
    counter.incrementPush();
}

private double calculate(double a, String operator, double b) {
    switch (operator) {
        case "+": return a + b;
        case "-": return a - b;
        case "*": return a * b;
        case "/":
            if (b == 0) {
                throw new ArithmeticException("หารด้วยศูนย์");
            }
    }
}

```

```

    }
    return a / b;
default:
    throw new IllegalArgumentException("operator ไม่ถูกต้อง: " + operator);
}
}
}

```

```

ป้อนนิพจน์ (Infix): 3 + 4 * 2 / (1 - 5)
--- Algorithm B ---
Postfix: (ไม่ได้สร้าง Postfix ใน Algorithm B)
ผลลัพธ์: 1.0
เวลาที่ใช้: 399600 ns
จำนวน Operation: Push=14, Pop=14, Comparisons=8, Loops: 11

```

Main.java:

คลาสหลักของโปรแกรม มีเมนูให้เลือกรัน Algorithm A หรือ B พร้อมแสดงผลลัพธ์ เวลาที่ใช้ และจำนวน Operation

```

import algorithms.AlgorithmA;
import algorithms.AlgorithmB;
import models.ExpressionResult;

import java.util.Scanner;

public class Main {

    public static void main(String[] args) {
        Scanner scanner = new Scanner(System.in);
        AlgorithmA algorithmA = new AlgorithmA();
        AlgorithmB algorithmB = new AlgorithmB();

        boolean running = true;

        while (running) {
            printMenu();
            String choice = scanner.nextLine().trim();

            switch (choice) {
                case "1":
                    System.out.print("ป้อนนิพจน์ (Infix): ");
                    String exprA = scanner.nextLine();
                    runAlgorithm("Algorithm A", algorithmA.evaluate(exprA));
                    break;

```



```

        case "2":
            System.out.print("ป้อนนิพจน์ (Infix): ");
            String exprB = scanner.nextLine();
            runAlgorithm("Algorithm B", algorithmB.evaluate(exprB));
            break;

        case "0":
            running = false;
            System.out.println("ออกจากโปรแกรม");
            break;

        default:
            System.out.println("กรุณาเลือกเมนูที่ถูกต้อง");
    }
}

scanner.close();
}

private static void printMenu() {
    System.out.println();
    System.out.println("Expression Processor: ");
    System.out.println("1. Algorithm A (Infix -> Postfix -> Evaluate)");
    System.out.println("2. Algorithm B (Direct Infix Evaluate)");
    System.out.println("0. ออกจากโปรแกรม");
    System.out.print("เลือกเมนู: ");
}

private static void runAlgorithm(String label, ExpressionResult result) {
    System.out.println("--- " + label + " ---");
    if (result.isSuccess()) {
        System.out.println("Postfix: " + result.getPostfix());
        System.out.println("ผลลัพธ์: " + result.getValue());
    } else {
        System.out.println("เกิดข้อผิดพลาด: " + result.getErrorMessage());
    }
    System.out.println("เวลาที่ใช้: " + result.getElapsedTimeNanos() + " ns");
    System.out.println("จำนวน Operation: " + result.getCounter());
}
}

```

=== MANDATORY TEST CASES ===

Input	Algorithm A	Algorithm B

3 + 4 * 2	11	11
(3 + 4) * 2	14	14
((8 + 2) * 5)	50	50
(3 + 4	ERROR	ERROR
3 + 4)	ERROR	ERROR
3 + * 4	ERROR	ERROR
10 / (5 - 5)	ERROR	ERROR
	ERROR	ERROR
3 + 4 * 2 / (1 - 5)	1	1

Size (n)	Algorithm	Avg Time (ns)	Push	Pop	Comparisons	Loop Iter	Data Moved	Est. Memory(KB)

101	Algorithm A	834,300	151	151	70	202	101	278.21
101	Algorithm B	203,780	151	151	71	101	0	123.78

1001	Algorithm A	1,647,620	1,501	1,501	767	2,002	1,001	1,238.35
1001	Algorithm B	670,600	1,501	1,501	769	1,001	0	992.05

10001	Algorithm A	8,474,200	15,001	15,001	7,539	20,002	10,001	1,532.66
10001	Algorithm B	3,614,760	15,001	15,001	7,540	10,001	0	2,410.38

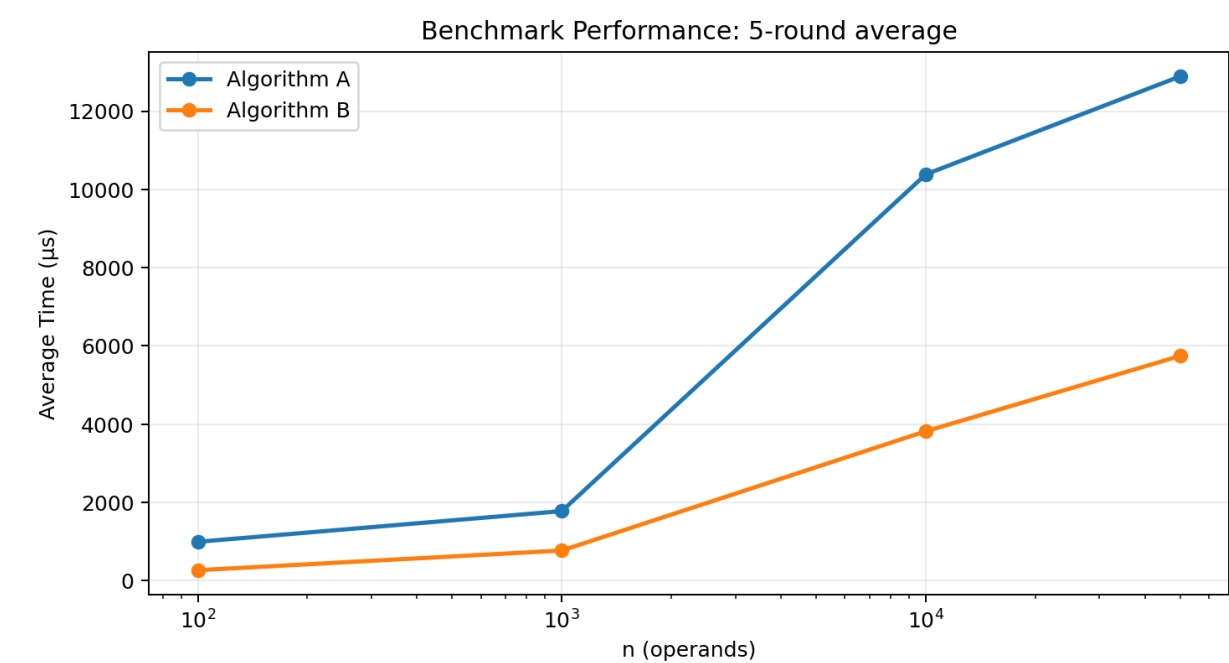
50001	Algorithm A	12,280,660	75,001	75,001	37,527	100,002	50,001	17,614.67
50001	Algorithm B	6,934,660	75,001	75,001	37,528	50,001	0	19,417.35

การทดลอง

ทำการทดลองเปรียบเทียบ Algorithm A และ Algorithm B ที่ขนาดข้อมูล $n = 100, 1,000, 10,000$ และ $50,000$ โดยเฉลี่ยผลจากการทดลอง 5 รอบต่อขนาดข้อมูล

n (operands)	Actual tokens	A Avg (μ s)	B Avg (μ s)	A Push/Pop	B Push/Pop	A Comp/Loop	B Comp/Loop
100	101	993.810	267.671	151/151	71/101	70/202	71/101
1,000	1,001	1,778.052	770.611	1,501/1,501	769/1,001	767/2,002	769/1,001
10,000	10,001	10,384.459	3,818.614	15,001/15,001	7,540/10,001	7,539/20,002	7,540/10,001
50,000	50,001	12,889.745	5,744.386	75,001/75,001	37,528/50,001	37,527/100,002	37,528/50,001

รูปที่ 7.1 กราฟเปรียบเทียบเวลาเฉลี่ยของ Algorithm A และ Algorithm B



อภิปรายผล: จากชุดข้อมูลนี้ Algorithm B ใช้เวลาเฉลี่ยน้อยกว่า Algorithm A ในทุกขนาดข้อมูล เพราะประเมิน Infix โดยตรงและไม่ต้องสร้าง Postfix เป็นผลลัพธ์กลาง อย่างไรก็ตามทั้งสองวิธีมีแนวโน้มเวลาเพิ่มตามขนาดข้อมูลและสอดคล้องกับ Time Complexity $O(n)$ ในเชิง Asymptotic. จำนวน Push/Pop และการวนรอบเพิ่มขึ้นเป็นสัดส่วนกับจำนวน Token ซึ่งสนับสนุนการวิเคราะห์เชิงทฤษฎี

หมายเหตุ: ในการทดลองนี้ n หมายถึงจำนวน Operand และ Actual tokens = $2n - 1$ สำหรับรูปแบบข้อมูลที่ใช้ทดสอบ

บันทึกการใช้ Generative AI:

- วิเคราะห์ปัญหา: ข้อสมมติฐาน, กรณีปกติ, กรณีขอบเขต, กรณีทำให้เกิด

Error

- Algorithm A, B + Pseudocode

- ตัวอย่าง Step-by-step ให้ออกมาเป็นตาราง

- อธิบายความถูกต้อง Algorithm A ทั้งหมด

- วิเคราะห์ประสิทธิภาพ

- ตารางสรุป Complexity

- วิเคราะห์ฟังก์ชัน

- Auxiliary Space

- เปรียบเทียบ A vs B

- Test Cases

- เขียนโปรแกรม Java ทั้งหมด